

LangChain Middleware vs Pi 执行管线对比

统一 Middleware 抽象 vs 三层 Hook 架构

技术架构白皮书 · 2026年6月

执行摘要

LangChain v1 引入了 `AgentMiddleware` 统一抽象，将所有代理扩展能力（观测、拦截、工具注册、状态管理）集中到一个类中。Pi 没有 "Middleware" 概念名，但其三层架构（Agent Loop 回调 + AgentEvent 流 + Harness/Extension Hook）实现了等价甚至更细分的能力。本文档从架构设计、钩子映射、拦截能力、组合模式四维度进行系统对比，揭示两种设计哲学背后的权衡与适用场景。

一、问题定义

Agent 框架需要一个标准化的扩展机制，允许开发者在不修改框架核心代码的情况下实现：

- 观测 — 日志、监控、追踪 Agent 执行过程
- 修改 — 改写消息、替换模型、注入指令
- 阻断 — 拦截危险的工具调用或 LLM 响应

- 4. 注册 — 向 Agent 添加额外的工具/资源
- 5. 控制 — 决定 Agent 的循环终止条件

核心问题: LangChain 的 AgentMiddleware 在 Pi 中对应哪些部分? 它们的共同之处和本质差异在哪里?

二、架构设计对比

2.1 宏观架构

维度	LangChain Middleware	Pi 三层架构
核心抽象	AgentMiddleware 统一类(12 个钩子方法)	三层: AgentLoopConfig + AgentEvent + ExtensionRunner
钩子类型	生命周期钩子(before/after)+ 拦截器(wrap)+ 工具注册 + 状态 schema	Loop 回调(控制流)+ Event(观测)+ Harness HookEvent(拦截)+ Extension 工具注册
组合方式	图节点链(生命周期)+ 洋葱闭包(拦截器)	内联函数调用(Loop)+ 多播通知(Event)+ 顺序管道&短路(Harness)
执行引擎	LangGraph(图执行)	纯函数式 ReAct Loop
类型系统	Python 泛型(StateT, ContextT, ResponseT)	TypeScript 联合类型 + 泛型 + Phantom Type
创建方式	子类化 + 7 个装饰器	设置 Agent 属性 + agent.subscribe() + ExtensionFactory
热插拔	不支持	Extension 层支持 reload

2.2 层级对应关系

关键发现: LangChain Middleware 的 12 个钩子方法 分散映射到 Pi 的三个独立层次, 没有一个 Union 层面的等价物。

LC Middleware 能力	Pi 等价系统	对应关系
------------------	---------	------

数量			
生命周期钩子 (before/after_model, before/after_agent)	8 个方法	Agent Loop 层 + Harness events	transformContext 、 convertToLlm 、 before_agent_start 等
拦截器 (wrap_model_call, wrap_tool_call)	4 个方法	Harness 层 (HookEvent) + Loop 层 (beforeToolCall)	context 、 before_provider_request 、 tool_call 、 tool_result 等
工具注册 (tools)	1 个属性	Extension 层	registerTool()
状态 schema (state_schema)	1 个属性	Agent 直接持有 + Extension rootState()	Agent._state , ExtensionAPI.rootState()
流变换器 (transformers)	1 个属性	无直接等价	Pi 通过 onPayload / onResponse 回调处理
装饰器 (@before_model 等)	7 个装饰器	无直接等价	Pi 的 AgentLoopConfig 回调通过对象属性赋值

一句话: LangChain 把能力集中到一个类, Pi 把能力拆到三个系统中。LC 的 Middleware 同时承担了 Pi 的 Loop 层、Event 层、Extension 层的职责。

三、钩子方法逐一映射

3.1 生命周期钩子映射

LC 方法	Pi 等价	实现方式
-------	-------	------

触发时机			
before_agent	Agent 执行开始前(一次)	before_agent_start Harness 事件	Extension 注册 on("before_agent_start") , 可修改 systemPrompt、注入初始消息
after_agent	Agent 执行完成后(一次)	agent_end AgentEvent + Harness 事件	agent.subscribe() 观测; Extension 处理器不参与控制
before_model	每次 LLM 调用前	transformContext + convertToLlm + before_provider_request	Loop 回调(消息级修改)+ Harness 事件(请求级修改)
after_model	每次 LLM 调用后	after_provider_response Harness 事件	Extension 管道, 在 LLM 响应完成后 emit

3.2 拦截器映射

LC 方法	能力	Pi 等价	能力对比
wrap_model_call	拦截 LLM 调用: 可重试、短路、修改请求/响应	context + before_provider_request + before_provider_payload + after_provider_response 组合	⚠️ Pi 可修改请求和响应, 但不能重试 handler(重试是 provider 层的事)
wrap_tool_call	拦截工具调用: 可重试、短路、修改请求/返回	beforeToolCall (Loop) + afterToolCall (Loop) + tool_call (Harness) + tool_result (Harness)	✅ 功能等价, 可以 block / patch 结果

关键差异: LC 的 `wrap_model_call` 通过洋葱闭包模式可以多次调用 `handler(request)` 实现重试。Pi 的 LLM 调用封装在 `streamFn` 闭包中, 外部无法获取 `handler` 引用。Pi 设计上认为重试是 `provider` 层的事(`SimpleStreamOptions.maxRetries`), 不是扩展层该管的。

3.3 工具注册和状态映射

LC 能力	Pi 等价	实现方式
<code>tools</code> 属性(Middleware 注册工具)	<code>ExtensionAPI.registerTool()</code>	扩展注册工具, <code>_refreshToolRegistry()</code> 合并到 agent 的工具列表
<code>state_schema</code> (Middleware 自定义状态)	<code>rootState() + Agent._state</code>	扩展通过 API 读写根状态; Agent 直接持有 <code>_state: MutableAgentState</code>
<code>transformers</code> (流变换器链)	无直接等价	Pi 通过 <code>onPayload / onResponse</code> 在 <code>provider</code> 层做流处理, 无通用 <code>transformer</code> 链

四、组合模式对比

4.1 洋葱包装 vs 链式管道

维度	LangChain 洋葱包装	Pi 链式管道
实现	<code>outer(inner(core_handler))</code> — 右到左包装成闭包链	<code>for h in handlers: result = await h(event)</code> — 顺序遍历
调用栈	完整的洋葱调用栈, 外层包裹内层	扁平顺序, 下游处理器看不到上游的修改(除非原地修改 <code>event</code>)
handler 控制	每个 Middleware 自主决定是否/何时/如何调用 handler	没有 handler 概念, 每个处理器只是处理一个事件
短路机制	不调用 handler → 链中断	返回 <code>cancel: true</code> 或 <code>block: true</code> → 剩余处理器跳过
重试能力	✓ 多次调用 handler 实现重试	✗ 无法重试(没有 handler 可调用)

LangChain 洋葱包装实现

```
# factory.py:221
def _chain_model_call_handlers(mw_list, core_handler):
    handler = core_handler
    for m in reversed(mw_list):          # 从右向左包装
        handler = make_handler(m, handler)
    return handler
# 结果: m[0](m[1](m[2](core_handler)))
```

Pi 链式管道实现

```
// runner.ts:693
async emit(event) {
    for (const ext of this.extensions) {
        const handlers = ext.handlers.get(event.type);
        for (const handler of handlers) {
            const result = await handler(event, ctx);
            if (result.cancel) return result; // 短路
        }
    }
}
```

4.2 生命周期钩子编排

维度	LangChain 图节点	Pi 内联调用
编排方式	每个 before/after hook 是独立的 LangGraph 节点, 用边连接	在 streamAssistantResponse 和 executeToolCalls 中直接 await 回调
执行顺序	START → before_m[0] → ... → before_m[n] → model → after_m[n] → ... → after_m[0]	在函数体中按书写顺序 await
条件跳转	✔ 通过 @hook_config(can_jump_to=["end", "tools", "model"]) 条件跳转	✔ 通过 shouldStopAfterTurn 终止、prepareNextTurn 修改下一轮
开销	每个 hook 是一个独立的图节点 trip	零开销内联调用

可
观
测
性

图节点自动记录到 LangSmith

需要显式 emit 事件到 Harness 层

五、代码示例：同一功能对比

5.1 重试失败的模型调用

LangChain(洋葱包装重试)

```
class ModelRetryMiddleware(AgentMiddleware):
    def wrap_model_call(self, request, handler):
        for attempt in range(3):
            try:
                return handler(request) # 可多次调用
            except Exception:
                if attempt == 2: raise
                time.sleep(2 ** attempt)
```

Pi(provider 层重试)

```
// Pi 无法在 AgentLoopConfig 层面重试 LLM 调用
// 重试由 provider 层处理:
// SimpleStreamOptions.maxRetries = 2
// 如果需要自定义重试逻辑, 在 provider 注册时处理
// 设计哲学: 重试是基础设施问题, 不是扩展问题
```

5.2 阻断危险工具调用

LangChain

```
class SafeGuardMiddleware(AgentMiddleware):
    def wrap_tool_call(self, request, handler):
        if request.tool_call["name"] in ["rm", "delete"]:
            return ToolMessage(
                content="Blocked: tool not allowed",
                tool_call_id=request.tool_call["id"]
            )
        return handler(request)
```

Pi(Loop 层 + Extension 层都支持)

```
// Loop 层
agent.beforeToolCall = async ({ toolCall }) => {
    if (["rm", "delete"].includes(toolCall.name))
```

```

        return { block: true };
    };

    // Extension 层
    extension.on("tool_call", async (event, ctx) => {
        if (event.toolName === "rm") {
            const approved = await ctx.ui.confirm("Approve rm?");
            if (!approved) return { block: true };
        }
    });
};

```

5.3 消息上下文裁剪

LangChain(before_model 返回状态更新)

```

class ContextTrimMiddleware(AgentMiddleware):
    def before_model(self, state, runtime):
        msgs = state["messages"]
        if count_tokens(msgs) > MAX:
            summary = summarize(msgs[: -10])
            state["messages"] = [sys(summary)] + msgs[-10:]
            return {"messages": state["messages"]}

```

Pi(transformContext 直接返回消息)

```

agent.transformContext = async (messages) => {
    if (estimateTokens(messages) > MAX) {
        const summary = await summarize(
            messages.slice(0, -10)
        );
        return [systemMsg(summary), ...messages.slice(-10)];
    }
    return messages;
};

```

六、设计哲学对比

Pi — "拦截与修改框架"

- 按能力分层：控制流归 Loop、观测归 Event、拦截归 Extension
- 各自独立注册，互不耦合
- 不依赖特定执行引擎
- Extension 热加载/卸载
- 适合应用开发者：按需在任意层级注入代码，灵活度高

LangChain Middleware — "大一统扩展入口"

- 所有扩展点集成在一个基类中

- 子类化 + 装饰器两种创建方式
- 强依赖 LangGraph(图执行引擎)
- Middleware 顺序直接影响执行行为
- 适合库设计者: 一次性定义好 Middleware 列表, 传给 `create_agent()`

七、完整能力矩阵

能力	LangChain Middleware	Pi Loop 层	Pi Event 层	Pi Harness 层
观测执行流程	✔ 生命周期钩子 + 图节点追踪	✘	✔ <code>subscribe()</code> 全部事件	✔ <code>on("*_end")</code>
修改 LLM 请求	✔ <code>wrap_model_call</code> + <code>before_model</code>	✔ <code>transformContext</code>	✘	✔ <code>context</code> + <code>before_provider</code>
修改 LLM 响应	✔ <code>wrap_model_call</code> 返回修改	✘	✘	✔ <code>after_provider</code> + <code>message_end</code>
重试 LLM 调用	✔ <code>wrap_model_call</code> 多次调 handler	✘	✘	✘ (provider 层处理)
阻断工具调用	✔ <code>wrap_tool_call</code> 返回伪造结果	✔ <code>beforeToolCall</code> { <code>block: true</code> }	✘	✔ <code>tool_call { block: true }</code>
修改工	✔ <code>wrap_tool_call</code> 返回值	✔ <code>afterToolCall</code> { <code>patch</code> }	✘	

具 结 果	✓ tool_result { c isError }			
重 试 工 具 调 用	✓ wrap_tool_call 多次调 handler	✗	✗	✗
控 制 循 环 终 止	✓ jump_to="end" shouldStopAfterTurn	✓	✗	✗
注 册 工 具	✓ tools 属性	✗	✗	✓ registerTool
自 定 义 状 态	✓ state_schema 合并	✗ (直接操作 Agent._state)	✗	✓ rootState()
UI 交 互	✗	✗	✗	✓ ctx.ui.select/c input
热 插 拔	✗	✗	✓ add/remove listener	✓ reload extension
条 件 跳 转	✓ @hook_config(can_jump_to=[...])	✗	✗	✗

八、适用场景对比

8.1 选择 LangChain Middleware 的场景

- 统一扩展入口: 希望一个类搞定所有 Agent 扩展需求
- 图编排能力: 需要并行执行多个 hook、条件跳转等复杂编排
- **LangSmith** 集成: 自动追踪每个 hook 的执行
- **Python** 生态: 与 DataDog、Prometheus 等集成
- 快速原型: 装饰器一键创建 Middleware

8.2 选择 Pi 三层架构的场景

- 精细职责分离: 观测不干扰控制、拦截不影响观测
- 需要 UI 交互: Extension 有完整的 UI 交互 API
- 需要热插拔: Extension 可动态加载/卸载/升级
- 需要持久化: Session 持久化保证扩展可审计可恢复
- **TypeScript** 类型安全: 每个事件有精确的 Result 类型
- 插件生态: 文件系统扫描加载, 用户可安装/分享扩展包

九、一句话总结

LangChain Middleware = 大一统的“瑞士军刀”: 一个类解决所有扩展问题, 统一、直接、powerful。

Pi 三层架构 = 职责分离的“微服务”: Loop 层控执行、Event 层做观测、Extension 层管拦截——各司其职, 互不干扰。

两者不是竞争, 而是不同设计哲学的体现: **Monolith vs Modular**。LangChain 把能力合在 Middleware 里追求一站式体验; Pi 把能力拆到三层中追求灵活度和可组合性。在实际项目中, 可以根据复杂度需求选择或借鉴对方的设计思路。